RBE595: Hands-On Autonomous Aerial Robotics P5: The Final Race!

Shreyas Devdatta Khobragade
Department of Robotics Engineering
Worcester Polytechnic Institute
Email: skhobragade@wpi.edu

Brijan Vaghasiya
Department of Robotics Engineering
Worcester Polytechnic Institute
Email: bcvaghasiya@wpi.edu

***Abstract*—This paper presents an integrated autonomy pipeline for a quadrotor to (i) traverse three structured window obstacles using learned semantic segmentation and image-based visual servoing, (ii) transition to an unknown gap navigation task using optical flow (RAFT) and edge-based gap extraction, and (iii) execute an efficient return flight by caching, modifying, and replaying traversed waypoints. The system is implemented as a state machine with explicit timing instrumentation: simulation time is accumulated from a fixed integration step, wall-clock time is measured from epoch time, and per-frame inference latency is logged for the segmentation network. Window traversal relies on a DenseNet-U-Net segmentation model that produces a binary mask, from which the largest feasible black opening is localized using contour and hierarchy reasoning, including robustness to partial, edge-touching openings. A discrete-time visual servoing law maps pixel-center error between the detected opening and image center into incremental target pose updates in the drone's lateral and vertical axes, with anomaly rejection for temporally inconsistent detections. Once aligned, the vehicle advances forward in fixed sub-steps to pass the window and then re-centers to ensure subsequent windows remain within the field-of-view. After three successful window traversals, the pipeline switches to a RAFT-based optical flow stage and the flow visualization is processed via Canny edges and morphological closing to identify the largest enclosed gap region, whose centroid drives the same servoing-forward strategy. Finally, the vehicle turns 180° and returns through modified waypoints that reduce the alignment-to-pass distance, enabling faster and more stable backtracking. The approach provides a practical end-to-end perception control loop for structured obstacle traversal plus unknown gap handling, with explicit runtime accounting and reusable waypoint memory.**

***Index Terms*—Quadrotor navigation, visual servoing, semantic segmentation, DenseNet-U-Net, RAFT optical flow, gap detection, waypoint caching, simulation timing, closed-loop control.**

I. Introduction

Autonomous micro-aerial vehicles operating in cluttered environments must align precisely to constrained openings (e.g., windows) while maintaining stability and meeting real-time computation constraints. In photorealistic simulators, this challenge is amplified by appearance changes, partial views, and the need to couple perception outputs to dynamics-aware control. This work addresses a structured traversal problem (three window-like obstacles) followed by an "unknown gap" task and a return journey. We implement a unified pipeline that: (i) initializes the quadrotor from an instructor-provided pose and moves to a nominal centered pose to ensure all windows are visible, (ii) uses a trained DenseNet-U-Net segmentation inferencer to detect window structures, (iii) extracts the largest feasible opening and aligns to it using image-based servoing, (iv) transitions to RAFT optical-flow-based gap detection after three successful window passes, and (v) returns efficiently by saving and modifying waypoints collected during the forward traversal.

II. Methodology

A. System Architecture and State Machine

The mission is organized into phases:

1) **Initialization and start pose centering:** the drone is moved from the instructor-provided initial pose to a nominal centered pose (centered in the lateral axis and with $z = 0$) to place all three windows within the camera field-of-view.
2) **Window traversal (three windows):** per-frame segmentation, opening localization, alignment, forward pass, and re-centering.
3) **Unknown gap traversal (RAFT):** optical flow estimation, gap extraction from flow visualization, alignment, and forward pass.
4) **Waypoint modification and return:** cache aligned/passed poses, shorten alignment points toward pass points, execute a 180° yaw, and traverse the modified waypoints in reverse order.

Throughout all phases, the system logs timing signals and debug artifacts (RGB frames, masks, and overlays), enabling reproducible performance inspection.

B. Timing Instrumentation: Simulation Time, Wall Time, and Inference Time

To evaluate real-time feasibility and computational behavior, the system explicitly logs three timing quantities during execution: simulation time, wall-clock time, and segmentation inference time. These measurements provide insight into control-loop pacing, perception latency, and overall mission duration.

1) Simulation Time: Simulation time advances deterministically based on a fixed integration step of $dt = 0.01$ s. Over the complete mission including three window traversals, the unknown gap traversal, and the return path the maximum

accumulated simulation time reached approximately **1 minute and 15 seconds**. This value represents the total simulated flight duration required to complete the task under nominal control and perception behavior. Simulation time is used internally for controller integration, waypoint tracking, and trajectory generation, and is independent of computational delays introduced by perception or logging.

2) Wall-Clock Time: Wall-clock time is measured using system time from mission start to completion. For the full run, the total wall time was approximately **3 minutes and 12 seconds**. Wall time includes all computation overheads such as neural network inference, optical flow estimation, disk I/O for image saving, and visualization and video generation. Wall time is periodically reported at fixed intervals (every 10 seconds) to provide coarse-grained runtime feedback during execution. The discrepancy between simulation time and wall time highlights the computational cost of perception modules and logging within the closed-loop autonomy pipeline.

3) Segmentation Inference Time: For each rendered RGB frame during window traversal, the DenseNet-U-Net segmentation inference time is measured independently using wall-clock timestamps. Across the mission, observed per-frame inference latency ranged between:

- **Minimum:** ∼31 ms
- **Maximum:** ∼239 ms

This corresponds to an effective inference frequency ranging approximately from:

- **Best case:** ∼32 Hz
- **Worst case:** ∼4 Hz

The variability in inference time arises from several factors, including GPU utilization, image saving and disk I/O overhead, and concurrent execution of other perception modules such as segmentation and visualization. Despite this variability, the visual servoing loop remains stable due to the use of bounded control updates, temporal anomaly rejection in center detection, and a strict decoupling between simulation time progression and wall-clock execution. All experiments were conducted on a system equipped with **16 CPU cores**, an **NVIDIA-class A100 80GB GPU**, and **64 GB of RAM**.

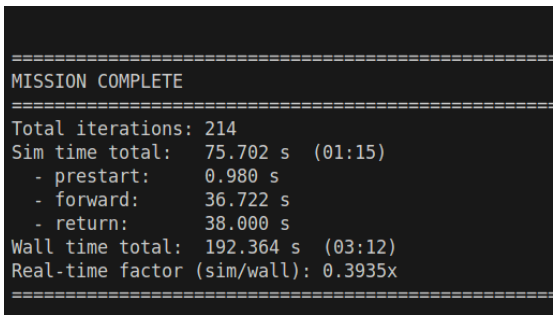

Fig. 1: Final Sim and Wall time after race completion

C. State Estimation Using Extended Kalman Filtering

In Extended Kalman Filter (EKF) was implemented to mitigate noise in the controller by filtering position and velocity states using a predict–update framework. The filter was integrated into the control loop and executed stably during flight.

However, in the simulated environment, the available state measurements were already relatively low-noise, making it difficult to conclusively evaluate the benefit of EKF filtering over direct state feedback. As a result, EKF outputs were not relied upon as a critical component of the final navigation or visual servoing logic. Nevertheless, the implementation serves as a foundation for future real-world deployment, where measurement noise and drift are expected to make state estimation more impactful. Although the EKF was successfully integrated and executed stably within the control loop, its effectiveness was difficult to evaluate conclusively in the simulation environment. Due to this uncertainty, the EKF estimates were not relied upon as a critical component of the final perception or control logic. However, the implementation provides a foundation for future real-world deployment, where sensor noise, drift, and intermittent measurements are expected to make state estimation significantly more impactful.

D. Perception Modules

1) DenseNet-U-Net Segmentation and Binary Masking: Let $I \in \mathbb{R}^{H\times W\times 3}$ be the RGB image. The segmentation network produces logits:

$$Z = f_\theta(I), \tag{1}$$

followed by a sigmoid to obtain per-pixel probabilities:

$$P = \sigma(Z). \tag{2}$$

A binary mask is computed by thresholding with τ:

$$M(u,v) = \mathbb{I}\{P(u,v) \geq \tau\}, \tag{3}$$

where $\mathbb{I}\{\cdot\}$ is the indicator function. Masks are resized to match the original image dimensions using nearest-neighbor interpolation to preserve label integrity.

2) Window Opening Localization from the Mask: The goal is to find the center of the largest feasible black opening (interior) between white frame regions. Two cases are handled:

a) Case A: complete windows (holes inside frames): Contours are extracted with hierarchy. Candidate openings are identified as child contours enclosed by a parent frame contour. Each candidate region $\mathcal{R}_i$ is scored, e.g., by area:

$$S_i = \text{Area}(\mathcal{R}_i), \tag{4}$$

and the selected opening is:

$$i^\star = \arg\max_i S_i. \tag{5}$$

The opening center is computed from region moments:

$$(u_w, v_w) = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right). \tag{6}$$

b) Case B: partial/edge-touching windows: If hierarchy-based holes are unavailable (e.g., the opening touches the image boundary), an inverted-contour strategy is used to detect feasible black regions that are not fully enclosed. The same area-based selection and moment-based centroid computation (Eq. (6)) is applied.

3) RAFT Optical Flow and Gap Extraction: After three window passes, the pipeline switches to an optical-flow-based gap detector using RAFT on consecutive frames. RAFT estimates a dense flow field:

$$\mathbf{F}(u, v) = (\Delta u(u, v), \Delta v(u, v)). \tag{7}$$

A flow visualization image is generated and processed to extract the largest gap region:

1) Apply Canny edge detection to obtain an edge map E.
2) Apply morphological closing to connect near-boundary segments and form closed contours.
3) Extract contours and select the largest enclosed region by area.
4) Fill the selected contour to obtain a binary gap mask G.
5) Compute the gap centroid $\mathbf{c}_g = (u_g, v_g)$ using moments (Eq. (6)).

E. Visual Servoing and Target Pose Updates

1) Window Servoing (Segmentation-Based): Let the image center be $\mathbf{c}_I = (u_0, v_0)$ and the detected opening center be $\mathbf{c}_w = (u_w, v_w)$. Define pixel error:

$$\mathbf{e} = \begin{bmatrix} e_u \\ e_v \end{bmatrix} = \begin{bmatrix} u_w - u_0 \\ v_w - v_0 \end{bmatrix}. \tag{8}$$

A discrete servo law maps pixel error to world-frame increments in lateral (y) and vertical (z) axes:

$$\Delta y = \mathrm{clip}(s\, e_u,\, -\Delta y_{\max},\, \Delta y_{\max}), \tag{9}$$

$$\Delta z = \mathrm{clip}(s\, e_v,\, -\Delta z_{\max},\, \Delta z_{\max}), \tag{10}$$

where s is a pixel-to-world scale factor and $\mathrm{clip}(\cdot)$ bounds per-step motion to maintain stability.

a) Alignment criterion: Alignment is declared if:

$$|e_u| \le \epsilon_u \quad \wedge \quad |e_v| \le \epsilon_v, \tag{11}$$

for tolerances ϵ_u, ϵ_v (e.g., 30 pixels).

b) Temporal anomaly rejection: To reduce sensitivity to spurious detections, the center measurement is checked against the last reliable center:

$$d_k = \|\mathbf{c}_w[k] - \mathbf{c}_{\mathrm{good}}[k-1]\|_2 . \tag{12}$$

If d_k exceeds a jump threshold, the system may temporarily reuse $\mathbf{c}_{\mathrm{good}}$ and/or reduce step sizes, while counting anomalies. After repeated anomalies, new detections are accepted to allow recovery from genuine viewpoint changes.

c) Forward pass and re-centering: Once aligned, the vehicle advances forward (positive x) in multiple sub-steps until a pass distance is reached, then transitions to a re-centering policy so the next window remains in view.

2) Gap Servoing (RAFT-Based): The RAFT stage reuses the same servoing structure, replacing $\mathbf{c}_w$ with the gap centroid $\mathbf{c}_g = (u_g, v_g)$. The alignment condition (Eq. (11)) and bounded updates (Eq. (10)) remain unchanged, enabling a unified "align then advance" strategy across both perception modes.

F. Motion Generation and Control

1) Waypoint Navigation and Trajectory Profile: Movement between poses is executed using a waypoint navigation routine with a fixed control step dt, position tolerance, maximum time cap, and a nominal speed parameter v. Let $\mathbf{p}_0$ be the current position and $\mathbf{p}_T$ the target. Define distance:

$$D = \|\mathbf{p}_T - \mathbf{p}_0\|_2. \tag{13}$$

A traversal time is selected:

$$T = \min\left(\frac{2D}{v}, T_{\max}\right). \tag{14}$$

A trapezoidal speed profile generates a scalar progression $p(t) \in [0, 1]$, and the reference trajectory is:

$$\mathbf{p}(t) = \mathbf{p}_0 + p(t)\,(\mathbf{p}_T - \mathbf{p}_0). \tag{15}$$

Velocity and acceleration references follow the motion direction and are provided to the controller at each time step.

2) Attitude Stabilization and Orientation Clipping: During window traversal, roll instability was observed, therefore, roll (and optionally pitch) were temporarily clipped (set to zero) to enforce stable viewing geometry. During the return phase, roll and pitch are set to zero and yaw is set to π (facing backward) to maintain consistent camera alignment while retracing the route.

3) Controller Structure and Actuation: The controller tracks the position/velocity/acceleration references and outputs rotor commands using a cascaded structure:

1) A position/velocity loop produces a desired net force and thrust.
2) A quaternion-based attitude error produces desired body rates.
3) A body-rate PID produces torque commands (τ_x, τ_y, τ_z).
4) A mixer maps thrust and torques into rotor commands, which are saturated to actuator limits.

This cascaded strategy allows smooth tracking of trapezoidal reference trajectories while respecting the quadrotor's dynamics.

G. Waypoint Caching, Modification, Turn-Around, and Return

During forward traversal, the system stores two poses per obstacle: *aligned* and *passed*. These waypoints are saved for reuse in the return trip.

To reduce time spent re-aligning during return, each aligned waypoint is shifted toward its corresponding passed waypoint:

$$\mathbf{p}'_{\mathrm{aligned}} = \mathbf{p}_{\mathrm{passed}} - \alpha\big(\mathbf{p}_{\mathrm{passed}} - \mathbf{p}_{\mathrm{aligned}}\big), \tag{16}$$

where $\alpha \in (0, 1)$ (e.g., $\alpha = 0.5$) retains only a fraction of the alignment distance. The return path is constructed by reversing the forward waypoint order. After passing the gap, the vehicle performs a yaw rotation:

$$\psi \leftarrow \mathrm{wrap}(\psi + \pi), \tag{17}$$

and then executes the reverse traversal through the modified waypoint list.

TABLE I: Representative Parameters Used in the Pipeline

Parameter	**Typical Value / Description**
Simulation step	$dt = 0.01\,\mathrm{s}$
Segmentation threshold	$\tau = 0.5$
Image alignment tolerance	$\epsilon_u = \epsilon_v \approx 30$ px
Pixel-to-world scale	$s \approx 7 \times 10^{-4}$
Waypoint nominal speed	$v \approx 0.1\,\mathrm{m/s}$
Waypoint tolerance	$\approx 0.018\,\mathrm{m}$
Return waypoint shorten factor	$\alpha \approx 0.5$
Return yaw	$\psi \leftarrow \psi + \pi$
Orientation clipping	roll/pitch set to 0 (stability)
Initial Position (x, y, z) [m]	[0.001, 0.0, 0.0]
Initial Orientation (roll, pitch, yaw) [°]	[0.0, 0.0, 10.0]

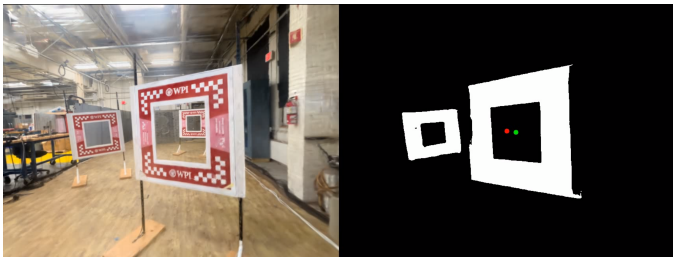

Fig. 2: Window1

III. Results

The system performs repeated cycles of segmentation inference, opening localization, visual servo alignment, and forward advancement to traverse three windows. After three successful pass events, the pipeline transitions to a RAFT-based gap detection stage. In this stage, dense optical flow is estimated between consecutive frames, and the largest enclosed gap is extracted using edge detection and morphological closing. The resulting gap centroid drives the same alignment-and-advance control policy used for window traversal, ensuring consistency across perception modes. All experiments were conducted using the parameters summarized in Table I.

IV. Limitations

Despite successful task completion in simulation, the proposed system has several limitations. The controller gains and nominal velocities require manual fine tuning, changes in forward speed can affect alignment accuracy, convergence rate, and stability. Similarly, the forward-advance distance used to pass windows and gaps is hand-tuned, and suboptimal values may lead to overshoot or insufficient progression. The gap

Fig. 3: Window2

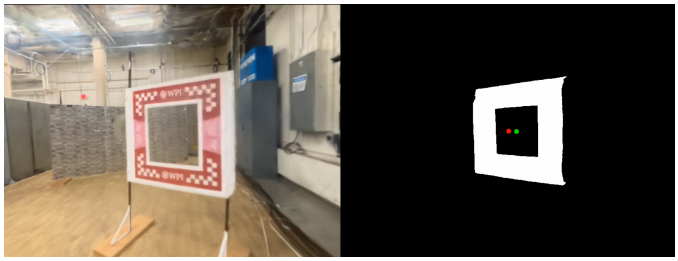

Fig. 4: Window3

Fig. 5: Unknown gap

Fig. 6: Rear view of Window 3

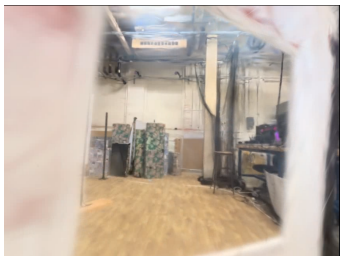

Fig. 7: Rear view of Window 2

Fig. 8: Rear view of Window 1

detection stage may intermittently fail when valid gaps are not detected in consecutive frames, which can temporarily disrupt the pipeline in the absence of stronger temporal tracking or confidence-based recovery. In addition, orientation stabilization relies on explicit roll and pitch clipping, which simplifies control but limits generality and dynamic maneuverability. Finally, the approach is evaluated entirely in simulation using workstation-grade compute, and performance under real-world sensing noise, limited compute, and tighter real-time constraints remains an open challenge.

V. Conclusion

We presented a complete perception-to-control pipeline for quadrotor traversal through multiple windows followed by an unknown-gap task, with a waypoint-optimized return. The approach couples: (i) learned DenseNet-U-Net segmentation and robust opening-center extraction (including partial-window handling), (ii) discrete-time image-based visual servoing with anomaly rejection and bounded increments, (iii) RAFT optical-flow-based gap localization using edge/morphology contour reasoning, and (iv) dynamics-aware waypoint tracking using trapezoidal trajectory references and a cascaded quaternion-based controller. The system further integrates comprehensive timing instrumentation (simulation time, wall time, per-frame inference latency) and uses waypoint caching/modification to accelerate the return route. This architecture provides a practical blueprint for structured obstacle traversal that remains extensible to additional obstacles, improved depth-aware scaling, and more advanced stability handling without orientation clipping.

VI. Team Contributions

The work was carried out collaboratively by two team members:

- **Brijan** developed the overall perception pipeline, integrated RAFT into the simulation framework, implemented Canny-based morphological contour gap extraction, applied filtering strategies, and contributed to return-path planning.
- **Shreyas** integrated the window-detection perception module within the VizFlyt simulator and SplatRenderer, conducted closed-loop visual servoing experiments in simulation, and contributed to return-path planning.

Acknowledgment

The authors thank the instructor, teaching assistant and dataset providers.

References

[1] RBE595 Project 2, "Documentation," *RBE595 Course Website*, Fall 2025. [Online]. Available: https://rbe549.github.io/rbe595/fall2025/proj/p2/

[2] RBE595 Project 3, "Documentation," *RBE595 Course Website*, Fall 2025. [Online]. Available: https://rbe549.github.io/rbe595/fall2025/proj/p3/

[3] RBE595 Project 4, "Documentation," *RBE595 Course Website*, Fall 2025. [Online]. Available: https://rbe549.github.io/rbe595/fall2025/proj/p4/

[4] RBE595 Project 5, "Documentation," *RBE595 Course Website*, Fall 2025. [Online]. Available: https://rbe549.github.io/rbe595/fall2025/proj/p5/

[5] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *Proc. Eur. Conf. Computer Vision (ECCV)*, 2020, pp. 402–419.

[6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, Jul. 2023.

[7] F. Chaumette and S. Hutchinson, "Visual Servo Control. I. Basic Approaches," *IEEE Robotics and Automation Magazine*, vol. 13, no. 4, pp. 82–90, Dec. 2006.

[8] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *Proc. ECCV*, 2020.

[9] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *Proc. CVPR*, 2017.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015.

[11] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.

[12] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[13] S. J. Maybeck, *Stochastic Models, Estimation, and Control*, vol. 1. New York, NY, USA: Academic Press, 1979.

[14] F. Chaumette and S. Hutchinson, "Visual Servo Control. I. Basic Approaches," *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, Dec. 2006.

[15] S. Hutchinson, G. D. Hager, and P. I. Corke, "A Tutorial on Visual Servo Control," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 5, pp. 651–670, Oct. 1996.